
Lab 02: Running a Linux Container with Docker on Your Machine

Course: Cloud Computing **Estimated Time:** 45–60 minutes **Difficulty:** Beginner (no prior Docker/Linux experience required)

1. Learning Objectives

By the end of this lab, you will be able to:

1. Install Docker Desktop on **Windows** or **macOS**.
 2. Verify your Docker installation is working correctly.
 3. Pull a Linux base image (Ubuntu) from Docker Hub.
 4. Run an interactive Linux container and execute basic commands inside it.
 5. Understand the difference between an **image** and a **container**.
 6. Stop, restart, and remove containers safely.
-

2. Why This Matters

Containers let you run a full Linux environment on top of Windows or macOS **without installing a virtual machine or dual-booting**. This is the foundation for almost everything in cloud computing — cloud providers run your applications inside containers, and tools like Kubernetes orchestrate thousands of them. Getting comfortable with Docker locally is the first step toward deploying containers in the cloud later in this course.

💡 A **container** is not a full virtual machine — it shares your host machine’s OS kernel but keeps its own filesystem, processes, and network, making it lightweight and fast to start.

3. Prerequisites

Before starting, make sure you have:

- ☐ A Windows 10/11 (64-bit) **or** macOS (Intel or Apple Silicon) laptop with **at least 8 GB RAM** and **10 GB free disk space**.
 - ☐ Administrator/root access on your machine (needed to install software).
 - ☐ A stable internet connection (you will download ~500 MB–1 GB of software and images).
 - ☐ **Windows only:** Confirm virtualization is enabled in your BIOS/UEFI (usually on by default). You can check this in Task Manager → Performance tab → CPU → “Virtualization: Enabled.”
-

4. Choose Your Operating System

Follow **Part A** if you use Windows, or **Part B** if you use macOS. Both paths converge at **Part C**, where the steps are identical.

5. Part A — Windows Setup

A1. Install WSL 2 (Windows Subsystem for Linux)

Docker Desktop on Windows relies on WSL 2 as its backend.

1. Open **PowerShell as Administrator** (right-click Start menu → *Terminal (Admin)* or *Windows PowerShell (Admin)*).
2. Run:

```
ws1 --install
```

3. Restart your computer when prompted.
4. After restart, a Linux terminal window (Ubuntu) may open automatically. If asked, create a UNIX username and password for WSL — this is separate from your Windows login and can be anything you like.
5. Verify WSL 2 is the default version:

```
ws1 --set-default-version 2
```

A2. Install Docker Desktop

1. Go to <https://www.docker.com/products/docker-desktop/> and download **Docker Desktop for Windows**.
2. Run the installer. Ensure **“Use WSL 2 instead of Hyper-V”** is checked during installation (this is the default on modern versions).
3. Once installation finishes, **restart your computer** if prompted.
4. Launch **Docker Desktop** from the Start menu.
5. Accept the license agreement and skip/sign in with a free Docker Hub account when prompted (recommended, so you can pull images without rate limits).
6. Wait until the Docker Desktop icon in the system tray shows **“Docker Desktop is running”** (a whale icon, no longer animating/loading).

✓ **Checkpoint:** Open **PowerShell** or **Windows Terminal** and continue to **Part C**.

6. Part B — macOS Setup

B1. Check Your Chip Type

1. Click the **Apple menu** → **About This Mac**.
2. Note whether it says **Apple Silicon (M1/M2/M3/M4)** or **Intel**. You’ll need this to download the correct installer.

B2. Install Docker Desktop

1. Go to <https://www.docker.com/products/docker-desktop/> and download **Docker Desktop for Mac** — choose the **Apple Silicon** or **Intel chip** version based on Step B1.
2. Open the downloaded `.dmg` file and drag the **Docker** icon into the **Applications** folder.
3. Open **Docker** from your Applications folder (macOS may ask for confirmation since it was downloaded from the internet — click **Open**).
4. Grant the requested **system permissions** (Docker needs privileged access to manage networking and file sharing).
5. Enter your **Mac login password** if prompted to allow Docker’s helper tools to install.
6. Sign in with a free Docker Hub account when prompted (recommended).
7. Wait until the whale icon in the top menu bar stops animating and shows **“Docker Desktop is running.”**

✓ **Checkpoint:** Open the **Terminal** app (Applications → Utilities → Terminal, or search via Spotlight with `Cmd + Space`) and continue to **Part C**.

7. Part C — Verify Your Installation (Windows & macOS)

Open your terminal (PowerShell/Windows Terminal on Windows, Terminal on macOS) and run:

```
docker --version
```

You should see output similar to:

```
Docker version 27.x.x, build xxxxxxxx
```

Next, confirm Docker can actually run containers:

```
docker run hello-world
```

This downloads a tiny test image and runs it. You should see a message beginning with:

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

✓ **Checkpoint:** If you see the “Hello from Docker!” message, your installation is working. If not, see the **Troubleshooting** table in Section 10.

8. Part D — Run a Linux Container Interactively

Now let’s pull a real Linux distribution and explore it.

D1. Pull the Ubuntu Image

```
docker pull ubuntu:22.04
```

This downloads the Ubuntu 22.04 image from Docker Hub. You can confirm it downloaded with:

```
docker images
```

You should see `ubuntu` listed with the `22.04` tag.

D2. Run an Interactive Container

```
docker run -it --name my-ubuntu ubuntu:22.04 /bin/bash
```

Flag breakdown: - `-it` → interactive mode with a terminal attached - `--name my-ubuntu` → gives the container a friendly name - `ubuntu:22.04` → the image to use - `/bin/bash` → the command to run inside the container (a Bash shell)

Your terminal prompt should now change to something like:

```
root@a1b2c3d4e5f6:/#
```

This means you are now **inside** a Linux container.

D3. Explore the Container

Try these commands inside the container:

```
cat /etc/os-release      # Confirm you're running Ubuntu
whoami                   # Should show 'root'
apt update                # Update the package list
apt install -y curl       # Install a small utility
curl --version            # Verify it installed
ls /                     # List root filesystem
exit                     # Leave the container
```

Running `exit` returns you to your host machine's terminal — the container stops running but still exists.

✓ **Checkpoint:** You successfully entered and exited a Linux container.

9. Part E — Managing Containers

Now that you've exited, practice basic container lifecycle commands:

Task	Command
List running containers	<code>docker ps</code>
List all containers (including stopped)	<code>docker ps -a</code>
Restart and reattach to your container	<code>docker start -ai my-ubuntu</code>
Stop a running container	<code>docker stop my-ubuntu</code>
Remove a container (must be stopped first)	<code>docker rm my-ubuntu</code>
Remove the downloaded image	<code>docker rmi ubuntu:22.04</code>
View disk space used by Docker	<code>docker system df</code>
Clean up unused data	<code>docker system prune</code>

Try this sequence:

```
docker ps -a
docker start -ai my-ubuntu
exit
docker stop my-ubuntu
docker rm my-ubuntu
docker images
```

✓ **Checkpoint:** Confirm with `docker ps -a` that `my-ubuntu` no longer appears in the list.

10. Troubleshooting

Issue	Likely Cause	Fix (Windows)	Fix (macOS)
Docker Desktop won't start	Virtualization disabled	Enable virtualization in BIOS/UEFI	N/A (not applicable on Mac)
<code>wsl --install</code> fails	WSL feature not enabled or outdated Windows	Update Windows via Settings → Update, then retry	N/A
"Docker daemon not running" error	Docker Desktop not fully started	Open Docker Desktop and wait for the whale icon to stop animating	Same
Permission denied running Docker commands	Docker Desktop not finished initializing, or user not in <code>docker-users</code> group	Restart Docker Desktop; ensure your Windows account is in the <code>docker-users</code> group	Restart Docker Desktop
<code>docker: command not found</code>	Terminal opened before install completed, or PATH not updated	Close and reopen PowerShell/Terminal	Close and reopen Terminal
Very slow image pulls	Free Docker Hub rate limits or network issues	Sign in to Docker Hub inside Docker Desktop	Same
Mac: "App can't be opened, unidentified developer"	macOS Gatekeeper security	Right-click the app → Open → confirm	—

11. Submission Checklist

Submit a short lab report including:

- ☐ Screenshot of `docker --version` output.
- ☐ Screenshot of the `hello-world` container output.
- ☐ Screenshot of `cat /etc/os-release` run **inside** the Ubuntu container.
- ☐ Screenshot of `docker ps -a` **before** and **after** removing `my-ubuntu`.
- ☐ One paragraph explaining, in your own words, the difference between a **Docker image** and a **Docker container**.

12. Key Terms Glossary

- **Image** – A read-only template (like a blueprint) containing an OS filesystem and application code, used to create containers.
- **Container** – A running (or stopped) instance created from an image; lightweight and isolated from the host OS.
- **Docker Hub** – A public registry for storing and sharing container images.
- **WSL 2 (Windows Subsystem for Linux)** – A compatibility layer that lets Windows run a real Linux kernel, used as Docker Desktop's backend on Windows.
- **Dockerfile** (*preview for a future lab*) – A text file with instructions to build a custom image.